Harness State Engineering for Reliable Agentic AI Systems

A Runtime-State-Centric Survey

ANONYMOUS AUTHOR(S)

Large language model (LLM) agents increasingly operate in long-horizon, tool-using, and multi-step environments where reliability depends not only on the underlying model but also on the execution harness surrounding it. Recent harness surveys have established the harness as an independent systems layer, while context engineering has clarified how information should be assembled for model inference. However, existing harness literature remains primarily architecture-, tooling-, or trajectory-safety-oriented. This survey proposes Harness State Engineering as a complementary runtime-state-centric perspective for reliable agentic systems. Whereas context engineering focuses on what information the model should see, Harness State Engineering focuses on what the runtime must track, control, validate, recover, and audit during execution. We synthesize the literature through seven state dimensions: execution, tool, memory/context, verification, observability, governance, and recovery state. We further propose a harness-state lifecycle, a maturity model, and an evaluation protocol that connect these dimensions to practical assessment.

Additional Key Words and Phrases: agentic AI, LLM agents, harness engineering, runtime state, observability, verification, governance, recovery, context engineering, AI safety

1 Introduction

The recent shift from single-turn prompting toward persistent, tool-using, and multi-agent execution has changed where the main engineering bottleneck lies in AI systems. Early LLM applications primarily depended on prompt engineering: the design of instructions, examples, and formatting constraints for a single model call. Context engineering extended this view by treating the full inference-time information payload as an object of engineering, including retrieval, memory, compression, tool context, and conversation state. As agentic systems increasingly plan, use tools, call APIs, modify files, coordinate sub-agents, and continue over many turns, reliability depends on more than prompts or context alone.

Harness engineering has emerged to describe the infrastructure surrounding an agent: execution environments, tool registries, context managers, state stores, lifecycle hooks, observability systems, verification interfaces, and governance controls. Recent work has made progress in formalizing this layer and showing that harness design can influence agent reliability even when the underlying model is unchanged. However, a key concern remains underdeveloped: the runtime state that the harness must maintain to make agent behavior controllable, recoverable, and auditable.

This survey proposes Harness State Engineering as a runtime-state-centric lens for reliable agentic AI systems. If context engineering asks what information the model should see, Harness State Engineering asks what the runtime must know, store, update, validate, recover, and audit while the agent acts. This framing is especially important in enterprise and regulated settings, where failures are not limited to incorrect final answers.

Our goal is not to claim that harness engineering itself is new. Existing surveys already study harness architectures, while recent empirical work examines automatic harness evolution and trajectory-level safety auditing. Instead, this survey contributes a state-centric synthesis around execution control, safe tool use, memory continuity, verification, observability, governance, and recovery.

2 Scope, Methodology, and Research Questions

This survey is structured as a narrative-systematic review. The scope covers LLM-based agentic systems that perform multi-step execution, interact with tools or environments, or require durable state across turns. We exclude work that

Table 1. Positioning Harness State Engineering against related work.

Literature Cluster	Primary Focus	Representative Contribution	Remaining Gap
Context Engineering	Inference-time information payload	Managing retrieved and compressed information	Does not fully address runtime control or recovery
Harness Surveys	Agent infrastructure	Execution, tools, lifecycle, observability, verification, governance	Often architecture-centric rather than state-centric
AHE	Automatic harness evolution	Observability-driven improvement of coding-agent harnesses	Focused on coding agents and evolution loops
HarnessAudit	Trajectory-level safety auditing	Permission boundaries, execution fidelity, stability	Safety-focused, not a full state taxonomy
This Survey	Runtime state engineering	Taxonomy, lifecycle, maturity model, evaluation protocol	Requires future empirical benchmarking

only studies single-turn prompting without external action, purely model-training methods without an execution harness, and application papers that do not expose reusable harness, state, evaluation, or governance concepts.

The literature was organized using seed papers on context engineering, agent harness surveys, observability-driven harness evolution, and trajectory-level harness safety. These were expanded with related work on LLM agents, tool use, agent benchmarks, memory systems, multi-agent orchestration, RAG, verification, observability, and governance. Papers were grouped by the primary runtime concern they address: execution, tools, memory/context, verification, observability, governance, recovery, or evaluation.

The review is guided by five research questions. RQ1: What runtime state must an agent harness maintain for reliable long-horizon execution? RQ2: How does Harness State Engineering differ from, and complement, context engineering and general harness engineering? RQ3: How do existing frameworks, benchmarks, and safety studies implicitly manage runtime state? RQ4: Which evaluation metrics assess state completeness, consistency, recoverability, auditability, and compliance? RQ5: What problems remain for standardizing and governing runtime state in enterprise agents?

This methodology avoids a chronological catalog of papers. Instead, the survey develops an original organizing framework: a seven-state taxonomy, a lifecycle model, a maturity model, and an evaluation protocol.

3 From Context Engineering to Harness State Engineering

Context engineering focuses on constructing the information payload available to the model at inference time. It includes selecting documents, retrieving relevant memory, compressing conversation history, ranking tool results, and ensuring that the model sees useful information under context-window constraints.

Harness engineering shifts attention from model input to runtime infrastructure. Agents do not merely answer; they act. They call tools, write files, execute code, query external systems, communicate with other agents, and continue across multiple steps. The harness defines what actions are exposed, how tool calls are routed, how failures are handled, how traces are collected, and how governance is enforced.

Harness State Engineering sits inside this broader harness layer but treats state as the primary object of analysis. A system may log a tool call but fail to record whether the call was authorized, which validation checks preceded it, whether the result was used in a final decision, or whether rollback remains possible.

4 Related Work Landscape

Existing work can be organized into five clusters. First, agent architecture and harness surveys formalize the execution layer around LLM agents. Second, LLM-agent research studies planning, tool use, memory, reflection, and environment interaction. Third, benchmark and evaluation work such as AgentBench, WebArena, SWE-bench, and related environments evaluates trajectories. Fourth, safety and governance work studies guardrails, policy compliance, information-flow risks, and human oversight. Fifth, context-engineering and RAG literature studies how external knowledge and memory are selected and presented to the model.

Harness State Engineering is complementary to these clusters. Planning requires execution state; tool use requires tool state; memory requires provenance and retention state; guardrails require verification and governance state; debugging requires observability state; and failure handling requires recovery state.

5 Formalizing Harness State

We define harness state at time t as

$$H_s(t) = (E_t, T_t, M_t, V_t, O_t, G_t, R_t),$$

where E is execution state, T is tool state, M is memory/context state, V is verification state, O is observability state, G is governance state, and R is recovery state. The tuple is a conceptual model for the minimum runtime information required to make an agent inspectable, governable, recoverable, and safe to evolve.

The state tuple separates architecture from state. Two systems may both contain a tool registry and a tracing system, but only one may record sufficient authorization, schema version, input provenance, validation result, and rollback state. Harness State Engineering therefore evaluates runtime state quality rather than merely checking component presence.

6 Seven Dimensions of Harness State

Execution state captures workflow progress: active step, task phase, planning branch, loop counter, termination condition, and checkpoint identity. Tool state captures tool availability, permissions, argument schemas, cached outputs, side effects, and error states. Memory/context state records what information has been retrieved, compressed, retained, discarded, or injected into the model context.

Verification state records whether intermediate or final actions satisfy constraints such as schemas, tests, policies, and business rules. Observability state stores traces, logs, metrics, timing information, cost indicators, and failure classifications. Governance state records permission boundaries, approval status, role bindings, information-flow restrictions, compliance requirements, and audit actors. Recovery state records checkpoints, rollback pointers, retry budgets, fallback paths, escalation triggers, and post-failure annotations.

7 Harness State Lifecycle

A state-centric survey should explain not only what state categories exist, but how they are created, checked, retained, and evolved during an agent run. We define the Harness State Engineering lifecycle as a recurring sequence: observe, represent, validate, commit, recover, audit, and evolve.

In the observe phase, the harness records signals from agent actions, tool calls, memory retrievals, validation checks, and policies. In the represent phase, signals are normalized into typed state objects. In the validate phase, the harness checks whether a proposed transition satisfies execution, tool, schema, and governance requirements. In the commit phase, accepted transitions are persisted. In the recover phase, state supports retry, rollback, resume, or escalation. In

Table 2. Harness State Engineering maturity model.

Level	Name	Capability
0	Implicit State	Runtime behavior is mostly captured as conversation or raw logs.
1	Logged State	Execution and tool calls are recorded for post-hoc inspection.
2	Typed State	Execution, tool, memory, and verification states are structured.
3	Governed State	Governance and recovery states support approval, rollback, escalation, and audit.
4	Evolvable State	State evidence drives governed harness improvement with versioning and reversibility.

Table 3. Evaluation metrics for Harness State Engineering.

Metric	What It Measures	Relevant State Dimension
State Completeness	Required runtime information is captured	Execution, Tool, Memory, Verification
State Consistency	State transitions remain coherent	Execution, Tool, Memory
Recoverability	System can resume, retry, rollback, or escalate	Recovery, Execution
Auditability	Actions and decisions can be reconstructed	Observability, Governance
Governance Compliance	Permissions and approvals are respected	Governance, Tool, Verification

the audit phase, transitions become inspectable. In the evolve phase, accumulated state evidence informs improvements to prompts, tools, middleware, verification rules, or governance policies.

8 Maturity Model

We propose a five-level maturity model. Level 0, Implicit State, describes systems where runtime behavior is mostly captured as conversation text or raw logs. Level 1, Logged State, records execution and tool calls for post-hoc inspection but offers limited recovery. Level 2, Typed State, structures execution, tool, memory, and verification states. Level 3, Governed State, makes governance and recovery first-class objects. Level 4, Evolvable State, uses state evidence to drive governed harness improvement with versioning and reversibility.

The maturity model reframes harness comparison from a component checklist into a reliability assessment: which runtime states are explicit, which transitions are validated, which failures are recoverable, and which changes are auditable.

9 Evaluation Protocol and Metrics

A state-centric framework is useful only if harnesses can be compared beyond final task success. We propose an evaluation protocol that treats the harness as the unit of evaluation. The benchmark task should specify intermediate milestones, tool boundaries, validation rules, recovery expectations, and governance constraints. During execution, the harness records state transitions across the seven state dimensions and evaluates both outcomes and trajectories.

Outcome-level metrics measure task completion and final-output validity. State-level metrics inspect runtime trajectories. We define five evaluation dimensions: state completeness, state consistency, recoverability, auditability, and governance compliance. These ask whether the harness records required information, maintains coherent transitions, supports rollback or escalation, enables reconstruction of actions, and respects permission boundaries.

10 Enterprise and Regulated-Domain Implications

Enterprise deployments introduce requirements often absent from research prototypes. In financial services, healthcare, legal operations, and software engineering, agents must operate under access controls, policy constraints, audit requirements, and human approval workflows. A correct final answer is insufficient if the path to that answer involved unauthorized access, missing validation, or unrecoverable side effects.

Harness State Engineering provides a vocabulary for these requirements. Governance state makes policy boundaries explicit. Verification state records compliance checks. Observability state supports incident investigation. Recovery state supports controlled failure and rollback. Memory/context state preserves provenance and sensitive-data handling. Tool state records side effects and authorization. Execution state enables continuity across long-running processes.

11 Open Research Challenges

Formal semantics for harness state transitions remain underdeveloped. Cross-framework state schemas are needed because current systems vary in how they represent traces, memories, tool calls, policies, and checkpoints. Scalable recovery remains difficult for long-horizon and multi-agent tasks because recovery must account for execution state, tool side effects, memory updates, and governance decisions.

Governed self-evolution is another challenge. AHE-like systems show that harnesses can improve through observability-driven evidence, but enterprise settings require versioning, approval, rollback, and accountability. Privacy-preserving observability is also open: rich state records may contain sensitive data, tool inputs, user context, or policy details.

12 Threats to Validity and Publication Integrity

This survey avoids unsupported quantitative claims. Where empirical values are not directly established in the cited literature, we use qualitative language rather than invented percentages or benchmark numbers. The proposed taxonomy is interpretive, and other taxonomies could organize the same literature by architecture, risk, lifecycle, or domain.

To reduce plagiarism risk, the manuscript uses original language and cites source papers for claims about existing work. Direct quotations are avoided unless necessary, and all definitions introduced here are framed as our synthesis. Before submission, authors should run an institutional similarity check and verify every BibTeX entry, citation, and claim. AI-assisted writing, if used, must comply with ACM policy and authors remain responsible for correctness and originality.

13 Conclusion

Harness engineering has become central to reliable agentic AI systems, but the next conceptual step is to study not only harness architecture, but also the runtime state that makes controlled execution possible. This survey proposes Harness State Engineering as that step. By organizing the field around seven runtime state dimensions and connecting them through a lifecycle, maturity model, and evaluation protocol, the survey provides a practical framework for reliable, recoverable, auditable, and governable agentic AI systems. The central takeaway is simple: context engineering optimizes what the model sees; Harness State Engineering optimizes what the runtime must know and manage while the agent acts.

References